WPI INTERACTIVE QUALIFYING PROJECT

#TeamGraduate Recommendations For Future Work

A SET OF RECOMMENDATIONS FOR FUTURE WORK ON #TEAMGRADUATE'S AI TOOLS

By Weaver Goldman, Ryan Pote, and Kyle Smith

Advised By Dr. Amanda Wittman and Dr. Erica Brozovsky

Sponsored By Helena Mboti, #TeamGraduate founder

ACADEMIC INSTITUTION

WPI

SUSTAINABLE DEVELOPMENT GOALS

4 QUALITY EDUCATION

10 REDUCED INEQUALITIES

PARTNER ORGANIZATION

TEAM graduate

#TeamGraduate AI Recommendations for Future Work

Weaver Goldman, Ryan Pote, Kyle Smith

In this document, we discuss our recommendations for future work on the #TeamGraduate AI WordPress plugin and AI server. While some design decisions are briefly justified here, we recommend reading our design document and main report to learn more about why different decisions were made.

Contents

1 Technical Future Work **1**

1.1 Performance and Scalability 1

1.2 Pedagogical Correctness 1

1.3 User Experience 2

1.4 AI and Data Processing 3

1.5 Infrastructure 3

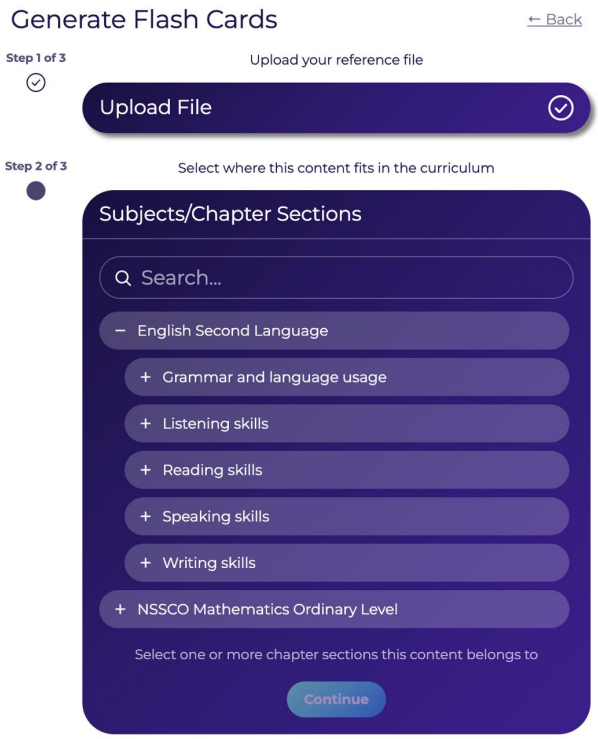
1 Technical Future Work

1.1 Performance and Scalability

The current #TeamGraduate AI system has many areas for potential performance improvement. For instance, when the user loads the uploaded curriculum to view it or select a specific lesson objective (see Figure 1), the system currently loads all curriculum nodes within the active curriculum. There are potentially hundreds of nodes per subject, and tens of subjects per national curriculum version.

Since the user will most likely only select from a single subject for each lesson, one potential improvement is to lazy-load at the top level. This will lead to a more complex search function, since the current search feature is implemented in the front end, where the entire curriculum hierarchy is available at once.

Figure 1: User Views Namibian Curriculum in #TeamGraduate AI Teaching Block



There are also many opportunities for caching on the AI server side. Right now, since the AI server is stateless, it cannot store persistent data. All persistent data is therefore stored in WordPress custom tables (MySQL) through the #TGAI plugin.

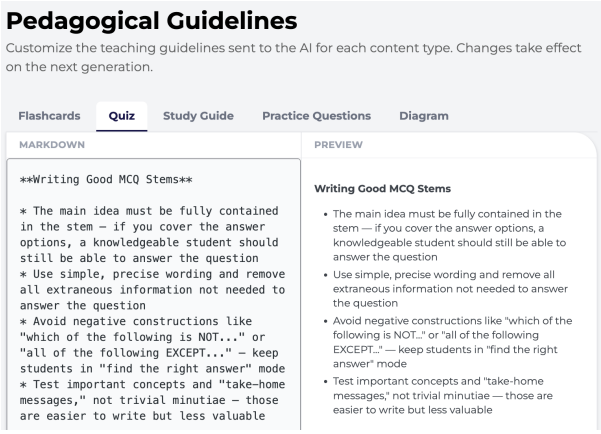
When a teacher generates new content, data is collected from the national curriculum, relevant textbooks, pedagogical (teaching) guidelines, and the teacher's lesson plan, aggregated, and sent to the AI server to help it create high-quality content. However, many data sources change very infrequently (i.e., the national curriculum, textbook content, teaching guidelines). It is possible to significantly reduce the amount of data sent to the AI server per generation request with caching.

1.2 Pedagogical Correctness

Currently, we do have a pedagogical guidelines section in the dashboard to inform the AI on how to write effective teaching content of different types, such as flashcards, practice quizzes, and study guides.

Through our discussions with members of #TeamGraduate, we identified a few additional areas to

Figure 2: Pedagogical (Teaching) Guidelines Editor in #TeamGraduate AI



affect pedagogical correctness. One of these is to allow teachers to provide their own personal teaching guidelines that will apply to all content they generate. We skipped this in our implementation since our sponsor wants consistent teaching guidelines across the website right now.

We also suggest allowing the teacher to specify grade level and Bloom's taxonomy levels to guide content generation. This allows for greater specificity.

Yet another possibility is running a second, more critical AI pass through the initial generated content to fix issues and align the content more closely with provided guidelines.

1.3 User Experience

Right now, any teacher or admin user can enter the dashboard and change settings or delete content. All users can also edit each other's generated content right now. There is more than enough information available to enable more fine-grained permissions.

It is also only possible to create content that is associated with the national curriculum. We recognize that a teacher may want to create custom content outside of this, so we recommending making this possible in the future. One way to do this is to add a virtual curriculum node that is automatically populated for all curriculum versions. Any orphan content can be tied to this node instead of the national curriculum.

On the student side, we suggest implementing spaced repetition for content types like flashcards. This is likely possible with the LearnDash integration, though you need to consider a number of details, like how the user will identify their comfort with a particular flashcard (i.e., again, good, easy) or if they can review all flashcards across lessons at once.

We also recommend adding teaching blocks that involve multiple students, like a matching game. The 2025 team worked on a game like this, and this may also be possible through the LearnDash integration (i.e., LearnDash manages the data), but if not, it is possible through custom WordPress tables, though this is less ideal.

We also suggest adding more extensive support for multimodal questions. Right now, the AI can generate most content with markdown formatting, including LaTeX and basic geometric shapes. However, adding diagrams, flowcharts, and other things to this could be extremely powerful.

We also suggest adding more on-demand AI features. Currently, when making flashcards, if a teacher wants to add more flashcards beyond the generated ones, they must write them manually. Other services offer the ability to generate new flashcards on demand. This may also be reasonable for other content types as well.

When looking at the dashboard page for the #TeamGraduate AI plugin, you'll find delete buttons on most of the pages. However, some require you to scroll horizontally depending on your screen size.

Another idea that came up during sponsor discussions was if teachers should be able to save and edit content locally and then upload it again later. This could be useful, but since we already manage all of the content in the WordPress database through the plugin and block editors, we decided not to implement this feature.

The sponsor also ideally wants a watermark on generated content that can be exported. Right now, this would include study guides, worksheets, and diagrams.

Currently, only the worksheet and study guide content types have preview buttons. It could be useful to make this a more universal block feature.

The dashboard currently only has views for individual content types. We considered adding some kind of unified hierarchy, but that was beyond the scope we decided on given our time frame. We also felt that there were fewer situations where that view would be important. However, if you have time to do it, there could be situations where it is advantageous.

The diagram teaching block is primarily designed for copying to PowerPoint. This is because during our discussions with #TeamGraduate, they noted that teachers would also be delivering lectures through their system, and AI content generation can also be leveraged to generate content for these presentations. It would be complex, but it may be possible to have a PowerPoint editor inside the website for this purpose.

One more thing is visual and wording consistency. A lot of different components do similar things but have slightly different wording or different implementations. This is definitely a potential area for polishing the user experience.

1.4 AI and Data Processing

The current system for parsing uploaded teacher content, textbooks, and the Namibian national curriculum revolves around text-based file parsing. We highly recommend experimenting with LLM-based OCR parsing as well, since all of the content may have important image content as well.

I also highly suggest looking into Microsoft's markitdown library. We are using a mix of parsing libraries, but it is likely the quality would improve when using this library, and the code could be simplified.

1.5 Infrastructure

Since our services are hosted on GCP, an admin needs to log in to their GCP console to see usage info for the Cloud Run, Gemini, and the Artifact Registry. It would be ideal to be able to see relevant information directly in the plugin dashboard.

We also have very basic security. It would be smart to look into the best approaches for keeping everything secure.

There is also no automatic testing, linting, or formatting. There is built-in formatting for the plugin code, but this is not automated at all. We suggest adding a CI/CD GitHub actions workflow to implement these. Adding more comprehensive testing will also likely reveal edge cases we did not handle, since we focused more on building foundational functionality with our limited time.

On a very different note, when a user goes to delete generated content, it allows them to, and any block that referred to that content will show an error message that the content no longer exists. However, one solution to this could be to add a reference counter for each generated content, so it can only be deleted once that count reaches zero. It would decrease when the user deletes the block or selects a different content type.

One feature that could be **very important** is generated content versioning and forking. This goes along with the permissions discussion earlier. It is currently possible (in the database) for multiple flashcards to be generated from the same uploaded lesson plan. However, due to limited time, we simplified this so each lesson plan can only have one instance of each content type. With some changes, it would be possible for users to fork each other's content (the original content would be read only for other users). It could also be possible to track version history.